

DevOps Interview Questions & Answers

30 Structured Q&A — For Candidates with 0–3 Years of Experience

This guide covers foundational DevOps concepts, version control, CI/CD, containerization, orchestration, and cloud/monitoring fundamentals — organized by topic for quick review before interviews.

1. DevOps Fundamentals

Q1. What is DevOps?

DevOps is a set of cultural philosophies, practices, and tools that combines software development (Dev) and IT operations (Ops) to shorten the development lifecycle and deliver high-quality software continuously.

- Emphasizes collaboration between development and operations teams
- Automates manual processes across build, test, release, and deployment
- Relies on continuous integration, continuous delivery, and continuous monitoring
- Promotes a culture of shared ownership and fast feedback loops

Q2. What are the benefits of DevOps?

- **Faster delivery:** shorter development cycles and quicker time-to-market
- **Improved collaboration:** breaks down silos between Dev and Ops
- **Higher quality:** automated testing catches defects earlier
- **Increased deployment frequency:** smaller, more frequent, lower-risk releases
- **Faster recovery:** automated rollback and monitoring reduce downtime
- **Cost efficiency:** automation reduces manual effort and human error

Q3. Explain the Software Development Life Cycle (SDLC).

SDLC is a structured process used to plan, design, build, test, and deploy software systems.

- **Planning:** gather requirements and define project scope
- **Design:** create architecture and system design documents
- **Development:** write and build the actual code
- **Testing:** validate functionality, performance, and security
- **Deployment:** release the application to production
- **Maintenance:** monitor, patch, and update the live system

2. CI/CD Concepts

Q4. What is Continuous Integration (CI)?

CI is the practice of frequently merging code changes into a shared repository, where each merge triggers an automated build and test process.

- Detects integration issues early by testing small, frequent changes
- Typically triggered by a Git push or pull request
- Common tools: Jenkins, GitHub Actions, GitLab CI, CircleCI

Q5. What is Continuous Delivery (CD)?

- Extends CI by automatically preparing code for a release after it passes all tests
- Ensures the application is always in a deployable state
- Final deployment to production still requires manual approval

Q6. Difference between Continuous Delivery and Continuous Deployment?

- **Continuous Delivery:** code is automatically tested and packaged, but a human manually triggers the production release
- **Continuous Deployment:** every change that passes automated tests is released to production automatically — no manual approval step
- Continuous Deployment is a stricter, fully automated extension of Continuous Delivery

3. Version Control (Git)

Q7. What is Git and why is it used?

Git is a distributed version control system (VCS) used to track changes in source code during software development.

- Allows multiple developers to work on the same codebase simultaneously
- Maintains a complete history of changes for auditing and rollback
- Supports branching and merging for parallel feature development
- Works offline since every clone is a full repository copy

Q8. Explain Git Merge vs Git Rebase.

- **git merge:** combines branches by creating a new merge commit; preserves full branch history
- **git rebase:** moves/replays commits from one branch onto another, creating a linear history without a merge commit
- Merge is safer for shared branches; rebase produces a cleaner history but rewrites commit hashes, so avoid rebasing public/shared branches

Q9. What is a Pull Request?

- A request to merge code changes from one branch (or fork) into another, typically into main or develop
- Enables code review, automated CI checks, and discussion before merging
- Acts as a quality gate to prevent unreviewed code from entering the main branch

Q10. What are Git Branching strategies?

- **Git Flow:** uses long-lived develop and main branches, plus feature, release, and hotfix branches
- **GitHub Flow:** a simpler model — short-lived feature branches merged directly into main via pull requests
- **Trunk-Based Development:** developers commit small changes directly to a single trunk branch frequently, often behind feature flags
- **Release Branching:** dedicated branches created per release for stabilization

4. Jenkins & Pipelines

Q11. How does Jenkins work?

Jenkins is an open-source automation server used to build, test, and deploy software as part of a CI/CD pipeline.

- Monitors a source code repository (e.g., via webhook) for changes
- Triggers a configured job/pipeline when a change is detected

- Executes build steps using agents/nodes, which can run in parallel
- Reports build status and publishes artifacts or test results
- Extensible through a large plugin ecosystem (Git, Docker, Kubernetes, Slack, etc.)

Q12. What is a Jenkins Pipeline?

- A suite of plugins supporting implementation and integration of continuous delivery pipelines as code
- Defined in a Jenkinsfile, which is typically checked into source control
- Composed of stages (e.g., Build, Test, Deploy) and steps within each stage
- Supports parallel execution, retries, and post-build actions (success/failure notifications)

Q13. Difference between Declarative and Scripted Pipelines?

- **Declarative Pipeline:** uses a structured, predefined syntax (pipeline { stages { ... } }); easier to read and validate; recommended for most use cases
- **Scripted Pipeline:** uses Groovy code directly within a node { ... } block; more flexible and powerful but harder to maintain
- Declarative is best for standard workflows; Scripted suits complex, custom logic

5. Infrastructure as Code

Q14. What is Infrastructure as Code (IaC)?

IaC is the practice of managing and provisioning infrastructure through machine-readable configuration files rather than manual processes.

- Infrastructure becomes version-controlled, repeatable, and auditable
- Reduces configuration drift between environments
- Common tools: Terraform, AWS CloudFormation, Ansible, Pulumi

Q15. Why is Terraform preferred for infrastructure provisioning?

- **Cloud-agnostic:** supports AWS, Azure, GCP, and hundreds of other providers with one tool
- **Declarative syntax:** describes the desired end state rather than step-by-step instructions
- **Plan/apply workflow:** previews changes before they are executed, reducing risk
- **State management:** tracks real-world resources to detect and reconcile drift
- **Reusable modules:** promotes DRY, modular infrastructure code

Q16. What is Terraform State File?

- A JSON file (terraform.tfstate) that maps real-world infrastructure resources to the configuration
- Used by Terraform to determine what changes are needed during terraform plan and terraform apply
- Should be stored remotely (e.g., S3 + DynamoDB lock) in team environments to avoid conflicts and enable locking
- Contains sensitive data, so access should be restricted and encrypted

6. Docker & Containers

Q17. Difference between Docker Image and Container?

- **Docker Image:** a read-only, immutable template/blueprint containing application code, dependencies, and runtime
- **Docker Container:** a running (or stopped) instance of an image — a live, isolated process with its own writable layer
- Analogy: an image is a class; a container is an object instantiated from that class

Q18. How do you optimize Docker Images?

- Use minimal base images (e.g., alpine) to reduce image size
- Leverage multi-stage builds to discard build-time dependencies from the final image
- Combine related RUN commands to reduce the number of layers
- Order Dockerfile instructions to maximize layer caching (least-changing first)
- Use a .dockerignore file to exclude unnecessary files from the build context
- Avoid installing unnecessary packages and clean up package manager caches

7. Kubernetes

Q19. What is Kubernetes?

Kubernetes (K8s) is an open-source container orchestration platform used to automate deployment, scaling, and management of containerized applications.

- Schedules containers across a cluster of machines (nodes)
- Provides self-healing through automatic restarts, replacement, and rescheduling
- Supports horizontal scaling, load balancing, and rolling updates
- Manages configuration and secrets centrally

Q20. Explain Pod, Deployment, and Service in Kubernetes.

- **Pod:** the smallest deployable unit in Kubernetes; wraps one or more tightly coupled containers that share network and storage
- **Deployment:** manages a set of identical Pods, handling replica counts, rolling updates, and rollbacks
- **Service:** a stable network endpoint that exposes a set of Pods, providing load balancing and service discovery regardless of Pod IP changes

Q21. What happens when a Pod crashes?

- Kubernetes' kubelet detects the failure via health/liveness checks
- If managed by a Deployment/ReplicaSet, a new Pod is automatically scheduled to replace the crashed one, maintaining the desired replica count
- Restart behavior is governed by the Pod's restartPolicy (Always, OnFailure, Never)
- Repeated crashes trigger a CrashLoopBackOff state with exponential backoff delays

Q22. How does Kubernetes Service Discovery work?

- Each Service is assigned a stable DNS name and ClusterIP via Kubernetes' internal DNS (CoreDNS)

- Pods can reach a Service using its DNS name instead of tracking individual Pod IPs
- kube-proxy maintains network rules (iptables/IPVS) to route traffic to healthy backend Pods
- Environment variables are also injected into Pods for basic service discovery

Q23. What is an Ingress Controller?

- A component that manages external HTTP/HTTPS access to Services within a cluster
- An Ingress resource defines routing rules (host/path-based) to backend Services
- The Ingress Controller (e.g., NGINX, Traefik, AWS ALB Controller) implements those rules and provisions the underlying load balancer/proxy
- Enables TLS termination, name-based virtual hosting, and path-based routing from a single entry point

8. Cloud & AWS

Q24. Explain AWS VPC Architecture.

A Virtual Private Cloud (VPC) is an isolated, logically segmented network within AWS where resources are launched.

- Spans one AWS Region and is divided into subnets across multiple Availability Zones
- **Public subnets:** route traffic through an Internet Gateway
- **Private subnets:** have no direct internet route; outbound access via a NAT Gateway
- Route tables control traffic flow between subnets and external networks
- Security Groups and NACLs provide layered network access control

Q25. Difference between Security Groups and NACLs?

- **Security Group:** instance-level (or ENI-level) firewall; *stateful* — return traffic is automatically allowed; supports only allow rules
- **NACL (Network ACL):** subnet-level firewall; *stateless* — return traffic must be explicitly allowed; supports both allow and deny rules
- NACLs are evaluated in rule-number order; Security Groups evaluate all rules together

Q26. What is an IAM Role?

- An IAM Role is an AWS identity with permission policies that determine what actions are allowed, but without long-term credentials (access keys)
- Assumed temporarily by trusted entities — AWS services (EC2, Lambda), users, or external accounts — via the Security Token Service (STS)
- Avoids hardcoding credentials in applications; commonly attached to EC2 instances or Lambda functions

Q27. Explain the purpose of CloudWatch.

- AWS CloudWatch is a monitoring and observability service for AWS resources and applications
- Collects metrics (CPU, memory, network, custom application metrics)
- Aggregates and stores logs via CloudWatch Logs
- Triggers Alarms based on metric thresholds (e.g., to scale or notify via SNS)
- Supports dashboards for real-time visualization of system health

9. Monitoring

Q28. How do Prometheus and Grafana work together?

Prometheus and Grafana form a widely used open-source monitoring stack.

- **Prometheus:** a time-series database that scrapes and stores metrics by polling HTTP endpoints (/metrics) on a defined interval
- Supports PromQL for querying metrics and Alertmanager for alert routing
- **Grafana:** a visualization layer that connects to Prometheus (and other data sources) to build dashboards and graphs
- Together: Prometheus collects and stores the data; Grafana visualizes it and can trigger/display alerts

10. Troubleshooting & Practical Scenarios

Q29. How would you troubleshoot a failed production deployment?

A structured, methodical approach minimizes downtime and identifies root cause quickly.

- Check deployment/pipeline logs to identify the exact failure point (build, test, or deploy stage)
- Verify application logs and error messages on the affected servers/Pods
- Check recent changes — code diff, config changes, or dependency updates introduced in the failing release
- Validate infrastructure health: resource limits, connectivity, DNS, certificates
- Roll back to the last known good version if the issue can't be quickly resolved
- Reproduce the issue in a staging environment to confirm root cause
- Document the incident and add monitoring/tests to prevent recurrence

Q30. Explain your complete CI/CD pipeline from code commit to production deployment.

A typical end-to-end pipeline includes the following stages:

- **1. Code Commit:** developer pushes code to a feature branch and opens a pull request
- **2. CI Trigger:** a webhook triggers the CI tool (e.g., Jenkins, GitHub Actions) on push
- **3. Build:** source code is compiled and a Docker image is built
- **4. Automated Testing:** unit, integration, and static code analysis/security scans run
- **5. Artifact Storage:** the validated image is pushed to a registry (e.g., ECR, Docker Hub)
- **6. Deploy to Staging:** IaC tools (Terraform/Helm) provision or update the staging environment
- **7. Approval Gate:** manual or automated approval before production release
- **8. Deploy to Production:** rolling, blue-green, or canary deployment strategy applied via Kubernetes/Jenkins
- **9. Post-Deployment Monitoring:** CloudWatch/Prometheus & Grafana track health, with automated rollback on failure